

Scikit-learn Basics for ML

scikit learn basics for ml using python

If you're learning Machine Learning in Python, **scikit-learn (sklearn)** is the most important library to master. It provides ready-to-use algorithms and tools for data preprocessing, model training, evaluation, and deployment.

1. What is Scikit-learn?

Scikit-learn is an open-source Python library built on:

- NumPy
- Pandas
- SciPy
- Matplotlib

It helps build ML models with very little code.

Install:

```
Bash  
pip install scikit-learn
```

Import:

```
Python  
import sklearn
```

2. Typical Machine Learning Workflow

```
Collect Data  
↓  
Load Dataset  
↓  
Preprocess Data  
↓  
Split Train/Test  
↓  
Choose Model  
↓  
Train Model  
↓  
Predict  
↓  
Evaluate  
↓  
Improve Model
```

3. Common Modules

Module	Purpose
sklearn.datasets	Sample datasets
sklearn.model_selection	Train-test split, Cross Validation
sklearn.preprocessing	Scaling & Encoding
sklearn.linear_model	Linear Regression, Logistic Regression
sklearn.tree	Decision Trees
sklearn.ensemble	Random Forest, Gradient Boosting
sklearn.svm	Support Vector Machine
sklearn.neighbors	KNN
sklearn.cluster	K-Means
sklearn.metrics	Accuracy, RMSE, Confusion Matrix
sklearn.pipeline	ML Pipelines

4. Loading Dataset

Example:

```
Python
from sklearn.datasets import load_iris

iris = load_iris()

print(iris.data[:5])
print(iris.target[:5])
```

Output:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.0 1.4 0.2]
 ...
]
[0 0 0 0 0]
```

5. Features and Target

```
Python
X = iris.data
y = iris.target
```

Where

```
X → Input variables
y → Output variable
```

6. Train-Test Split

Very important step.

```
Python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

80% Training

20% Testing

7. Feature Scaling

Many algorithms require scaling.

```
Python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Notice:

```
fit()
learns mean/std
transform()
applies scaling
fit_transform()
does both
```

8. Training First Model

Logistic Regression

```
Python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)
```

Here

```
fit()
```

means training the model

9. Prediction

```
Python
predictions = model.predict(X_test)
print(predictions)
```

Probability prediction:

```
Python
model.predict_proba(X_test)
```

10. Accuracy

```
Python
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, predictions)
print(accuracy)
```

Example

```
0.97
```

97% accuracy.

11. Confusion Matrix

```
Python
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, predictions)
print(cm)
```

Useful for classification problems.

12. Classification Report

```
Python
from sklearn.metrics import classification_report
print(classification_report(y_test, predictions))
```

Shows:

- Precision

- Recall
- F1 Score
- Support

13. Linear Regression Example

```
Python
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

prediction = model.predict(X_test)
```

14. Model Evaluation (Regression)

```
Python
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, prediction)

print(mse)
```

Also useful:

```
Python
from sklearn.metrics import r2_score

r2 = r2_score(y_test, prediction)

print(r2)
```

15. Decision Tree

```
Python
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()

model.fit(X_train, y_train)
```

16. Random Forest

```
Python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
```

17. K-Nearest Neighbors

```
Python
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)

model.fit(X_train, y_train)
```

18. Support Vector Machine

```
Python
from sklearn.svm import SVC

model = SVC()

model.fit(X_train, y_train)
```

19. Cross Validation

Instead of one train-test split:

```
Python
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    model,
    X,
    y,
    cv=5
)

print(scores)
print(scores.mean())
```

20. Grid Search (Hyperparameter Tuning)

```
Python
from sklearn.model_selection import GridSearchCV

parameters = {
    "max_depth": [3, 5, 7],
    "min_samples_split": [2, 5, 10]
}

grid = GridSearchCV(
    DecisionTreeClassifier(),
    parameters,
    cv=5
)

grid.fit(X_train, y_train)

print(grid.best_params_)
```

21. Pipeline

Instead of manually scaling:

```
Python
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipe.fit(X_train, y_train)
```

22. Saving Model

```
Python
import joblib

joblib.dump(model, "model.pkl")
```

Load later

```
Python
model = joblib.load("model.pkl")
```

23. Important Algorithms

Classification

- Logistic Regression
- Decision Tree
- Random Forest
- SVM
- KNN
- Naive Bayes
- Gradient Boosting

Regression

- Linear Regression
- Ridge
- Lasso
- Decision Tree Regressor
- Random Forest Regressor

Clustering

- KMeans
- DBSCAN

- Agglomerative Clustering

24. Essential Functions to Remember

Function	Purpose
<code>fit()</code>	Train the model
<code>predict()</code>	Predict labels/values
<code>predict_proba()</code>	Predict class probabilities
<code>score()</code>	Quick model performance
<code>transform()</code>	Transform data
<code>fit_transform()</code>	Learn and transform
<code>inverse_transform()</code>	Revert transformed data

25. Complete Example

```

Python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load data
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))

```

Learning roadmap

1. Learn `train_test_split()`
2. Learn `StandardScaler`
3. Train models with `fit()`

4. Make predictions using `predict()`
5. Evaluate with `accuracy_score()`, `confusion_matrix()`, and `classification_report()`
6. Try different algorithms (Decision Trees, Random Forest, SVM, KNN)
7. Learn cross-validation and hyperparameter tuning with `GridSearchCV`
8. Build reusable workflows using `Pipeline`

Since you're preparing for **Data Analyst** and eventually **Data Scientist** roles, mastering these fundamentals will give you the foundation needed before moving on to advanced topics like feature engineering, ensemble methods (XGBoost/LightGBM), and model deployment.

ChatGPT can make mistakes. Check important info.